

Assignment 2: Data Modelling

Shambhavi Tewari

Task 1: Regression

The objective of this task is to build a Simple Linear Regression model to predict alcohol content using density as the independent variable. The model was fitted using a random sample of 200 data points, with a focus on examining the linear relationship between the two variables. The findings are displayed with a regression graph to help identify the relationship. The model's performance was assessed using relevant metrics like R-squared.

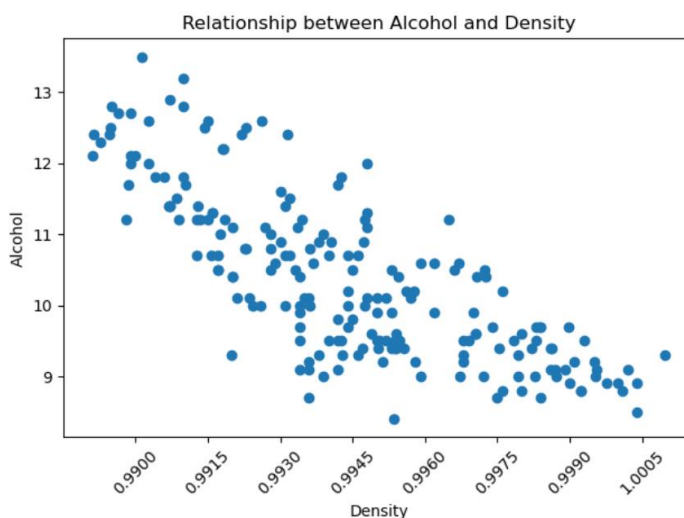


Figure 1: The plot showing relationship between Alcohol and Density.

Inference: The alcohol and density have a negative linear relationship. Alcohol concentration drops as density increases. The plot points typically follow a declining trajectory from left to right, making this pattern evident.

Simple Linear Model

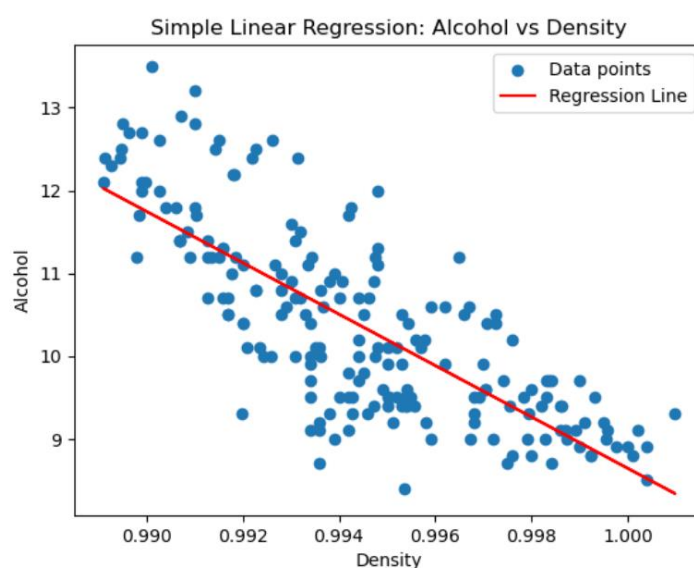


Figure 2: Plot showing the linear regression.

Relationship between Alcohol and Density: Alcohol content and density have a significant negative linear relationship, as seen by the scatter plot. Alcohol concentration decreases as density increases.

Interpretation of Coefficients:

Intercept (319.27): The alcohol content at zero density is represented by the intercept. According to the intercept in this linear regression model, the expected alcohol concentration would be 319.27 if the density were hypothetically zero.

Slope (-310.63): This means that the alcohol concentration drops by about 310.63 units for every unit rise in density.

Model Evaluation:

R-squared: 0.5992
 Mean Squared Error (MSE): 0.5366
 Root Mean Squared Error (RMSE): 0.7325
 Mean Absolute Error (MAE): 0.5743

Figure 3: Evaluation Metric.

Inference: The linear relationship with density accounts for 59.92% of the variability in alcohol content, according to the R-squared value of 0.5992. Even while this indicates a very strong linear relationship, density alone is not able to account for approximately 40.08% of the variability in alcohol concentration, suggesting that alcohol content may also be influenced by other factors. The RMSE (0.7325) and MSE (0.5366) show the average error between the actual and projected alcohol concentration. These values indicate a modest degree of prediction inaccuracy; lower RMSE and MSE values are preferable. MAE (0.5743) shows that the model's predictions differ by an average of 0.5743 units of alcohol concentration from the actual data. The model is better the closer this value is to zero.

The error metrics (MSE, RMSE, and MAE) indicate that the model does a reasonable performance for forecasting alcohol levels based on density. The model accounts for about 60% of the variability in alcohol content.

Task 2: Classification

The objective of this task is to implement and evaluate the k-Nearest Neighbours (kNN) algorithm. The task also aims to improve the performance of the kNN model with dimensionality reduction methods including feature selection and Principal Component Analysis (PCA). To evaluate the effect of these improvements, the performance of the original kNN model is compared with the revised models.

kNN

kNN Accuracy: 0.47				
	precision	recall	f1-score	support
3	0.00	0.00	0.00	1
4	0.00	0.00	0.00	4
5	0.41	0.56	0.48	64
6	0.54	0.57	0.55	92
7	0.46	0.18	0.26	33
8	0.00	0.00	0.00	5
9	0.00	0.00	0.00	1
accuracy			0.47	200
macro avg	0.20	0.19	0.18	200
weighted avg	0.46	0.47	0.45	200

Figure 4: Knn accuracy score.

Inference: With F1-scores of 0.48 and 0.55 for classes 5 and 6, respectively, the kNN model performed best, achieving 47% accuracy. However, it didn't perform well in underrepresented classes (e.g., 3, 4, 8, 9), where class imbalance results in 0.00 precision and recall. In contrast to the weighted average, which shows better overall results, particularly for the larger courses, the macro average scores show dismal performance in all classes. Improving model performance may involve addressing class imbalance and fine-tuning the parameters.

Modified kNN

kNN with PCA Accuracy: 0.508

	precision	recall	f1-score	support
4	0.00	0.00	0.00	7
5	0.54	0.53	0.53	83
6	0.51	0.65	0.57	110
7	0.46	0.28	0.34	40
8	0.00	0.00	0.00	9
9	0.00	0.00	0.00	1
accuracy			0.51	250
macro avg	0.25	0.24	0.24	250
weighted avg	0.48	0.51	0.48	250

Figure 5: Modified Knn accuracy score.

Inference: Using Principal Component Analysis (PCA), the kNN model's accuracy is 0.504, an improvement above the kNN model's initial accuracy of 0.47. This demonstrates a minor performance improvement brought about by PCA.

The kNN with PCA model's macro average accuracy, recall, and F1-score range from 0.26 to 0.41, suggesting that the model performs differently in each class, with the minority classes being more difficult to forecast.

The weighted average precision and recall of 0.50 and 0.50, respectively, show that the model works better for the classes that are more prevalent, such as Class 6, which probably has a bigger impact on the performance measures as a whole.

Comparison between Knn and Modified Knn

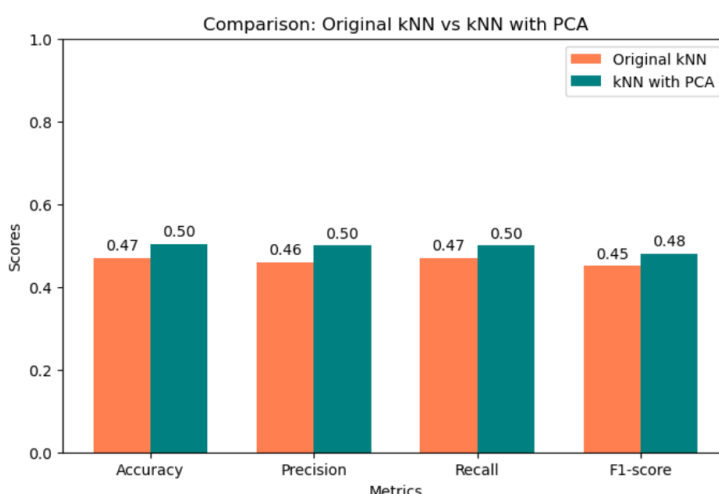


Figure 6: Comparison Plot of Knn and Modified Knn.

Inference: The comparison between the original kNN and kNN with PCA demonstrates that adding PCA has led to a small improvement in performance. The accuracy of the kNN with PCA is 0.50,

while the accuracy of the original kNN is 0.47. Other metrics including precision, recall, and F1-score show similar increases, with kNN with PCA marginally outperforming the original kNN.

Precision has increased from 0.46 to 0.50.

Recall has improved from 0.47 to 0.50.

F1-score has increased from 0.45 to 0.48.

This shows that, despite the relatively small gains, using PCA has improved the model's ability to identify patterns in the dataset.

Justification for Improved Performance with PCA:

By using PCA, the dataset's dimensionality is decreased, aiding in the removal of duplicated features and noise. Through this technique, the data are divided into a collection of most variance-retaining orthogonal (uncorrelated) components. Consequently, PCA minimizes the chance of overfitting and concentrates on the most crucial features, which is particularly helpful in kNN as the algorithm is vulnerable to dimensionality.

As evidenced by the measurements, PCA helps kNN concentrate on more pertinent patterns by lowering the number of features, which marginally enhances performance. This approach produces more balanced results across precision, recall, and F1-score when there is noise or correlation between features. However, the dataset might not really benefit from PCA, according to the minimal improvement, or major performance benefits would require additional preprocessing and tuning.

Decision Tree (and Comparison with Knn)

Fitting 5 folds for each of 24 candidates, totalling 120 fits

Best parameters found: {'criterion': 'gini', 'max_depth': 5, 'min_samples_split': 20}

Decision Tree Accuracy: 0.5480

	precision	recall	f1-score	support
4	0.50	0.14	0.22	7
5	0.58	0.72	0.64	83
6	0.53	0.63	0.57	110
7	0.50	0.17	0.26	40
8	0.00	0.00	0.00	9
9	0.00	0.00	0.00	1
accuracy			0.55	250
macro avg	0.35	0.28	0.28	250
weighted avg	0.52	0.55	0.51	250

Figure 7: Decision Tree accuracy score.

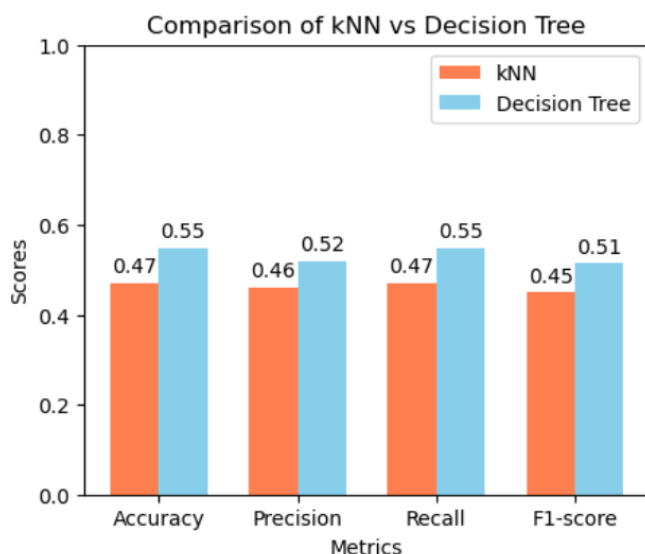


Figure 8: Decision Tree and Knn comparison plot.

Inference: With an accuracy of 0.5480, the Decision Tree classifier showed a slight improvement over kNN. The reason for this improvement can be related to the efficient selection of hyperparameters using GridSearchCV, which found the ideal combination of `max_depth=5`, `min_samples_split=20`, and `criterion='gini'`. Although the model was able to more accurately depict the dominating classes, but it was still unable to adequately represent the underrepresented classes.

The Decision Tree model performs better than kNN in all evaluation parameters, according to a comparison between the kNN and Decision Tree classifiers. The Decision Tree outperformed kNN with an accuracy of 0.55 as opposed to 0.47. With recall scores of 0.55 and 0.47, respectively, the Decision Tree outperformed kNN once more. Likewise, the Decision Tree's F1-score was 0.51 as opposed to kNN's 0.45.

The Decision Tree performs better than other algorithms because it can deal with class imbalances effectively. When parameters like `max_depth` and `min_samples_split` are set appropriately, decision trees perform exceptionally well at dividing the dataset space to suit majority and minority classes. This is crucial for models applied to imbalanced datasets because it enables the Decision Tree to obtain higher recall and F1-scores, particularly for the majority class (class 6). On the other hand, because kNN is based on proximity-based categorization, it often has trouble with datasets where distance metrics do not adequately specify the decision boundaries. In this instance, it was unable to match the Decision Tree's capacity to discriminate between classes, especially the minority ones. It functions better when there is a distinct distance-based difference between classes.

The Decision Tree is a preferable option for this dataset because to its ability to better balance recall and precision; nevertheless, overfitting must be avoided, especially with deeper trees.

Task 3: Clustering

K-Means

k	Inertia	Silhouette Score
2	2539.45	0.2206
3	2294.30	0.1590
4	2112.79	0.1623
5	1919.76	0.1402
6	1795.39	0.1604
7	1700.37	0.1343
8	1639.61	0.1373
9	1585.57	0.1209
10	1524.19	0.1211

Figure 9: Inertia and Silhouette score for different k.

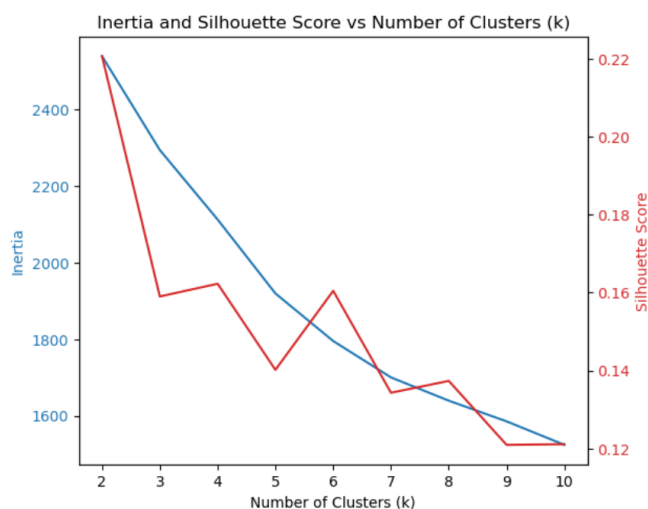


Figure 10: Silhouette Score vs Number of Clusters

Inference: As k increases, the inertia gradually reduces as the number of clusters decreases the sum of squared distances between data points and their designated centroids. When two clusters are utilized, the Silhouette Score is highest at $k=2$ (0.2206), indicating that the clusters are well-formed and separated. The silhouette score continuously drops with the number of clusters, indicating a worse degree of separation between clusters. $K=2$ seems to be the ideal number of clusters, where the clusters are the most distinct and well-formed, based on the balance between Inertia and the Silhouette Score.

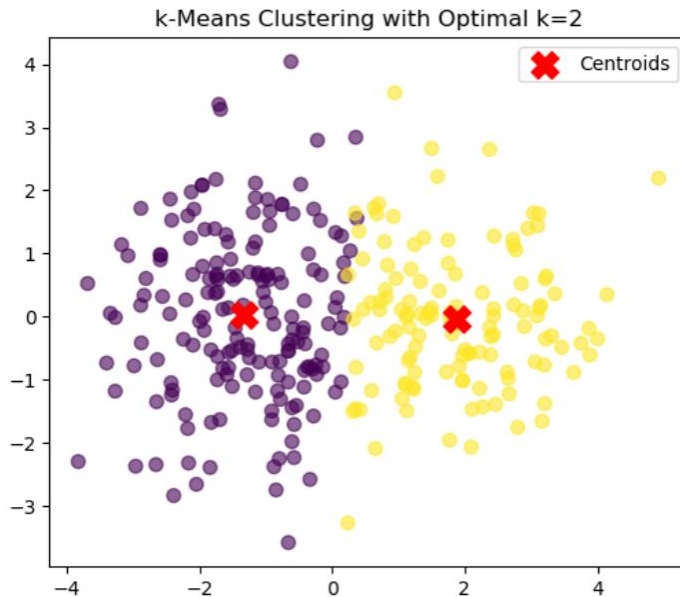


Figure 11: k-Means Clustering with Optimal $k=2$

Inference: Most of the points explicitly belong to one of the two clusters, which are substantially separated from one another. The silhouette score, which proposed that $k=2$ is ideal based on the clear boundaries between clusters, is validated by this. To get the optimum possible separation of the clusters, the algorithm iteratively changed the centroids (shown by red crosses), which show the mean position of all points within each cluster. The outcome clearly separates the dataset's two major groups, suggesting that $k=2$ adequately represents the underlying structure of the data.

Calinski-Harabasz Index for k-Means with $k=2$: 89.2497265712905
 Davies-Bouldin Index for k-Means with $k=2$: 1.7125654562221606

Figure 12: Performance Evaluation of K-means.

Inference: The ratio of the total between-cluster dispersion to the within-cluster dispersion is measured by the Calinski-Harabasz Index, also called the Variance Ratio Criterion. A higher score implies better-defined clusters. The clusters appear to be compact and well-separated, as indicated by the value of 89.25 for $k=2$, supporting the choice of $k=2$ as the ideal number of clusters.

The average similarity ratio between each cluster and its most similar cluster is measured by the Davies-Bouldin Index. Better grouping is implied by a lower score. The inter-cluster similarity of 1.71 for $k=2$ is rather low, indicating that while the clusters are distinct, they still have room for improvement. Although this score indicates reasonable separation, there is still opportunity for growth because values closer to 0 are optimal.

Limitations and Solutions

One of the fundamental drawbacks of k-Means is the assumption that clusters are spherical, which may not always hold true, as evidenced in the clustering results with some overlap in the visualization. Its inability to detect clusters of arbitrary shapes is a result of this strict supposition, which makes clustering less accurate when clusters are not well divided or have irregular forms. To solve this, an appropriate solution would be to switch to techniques such as DBSCAN or Gaussian

Mixture Models (GMM), which allow for more flexible cluster forms and can better handle overlapping clusters, resulting in more accurate clustering outputs.

The sensitivity of k-Means to outliers, which can severely skew the location of cluster centroids and result in inaccurate cluster assignments, is another important drawback. Because k-Means gives each data point the same weight, this is particularly troublesome in datasets with noisy or extreme data points. Applying k-Medoids rather than k-Means would be a solution to this problem because k-Medoids is more resilient to outliers. As opposed to k-Means, k-Medoids employs actual data points as the cluster centres, reducing the effect of outliers on the overall clustering outcomes and generating more dependable clusters.

DBSCAN

```
### DBSCAN with min_samples=3 ###
Eps: 0.1, MinPts: 3 - Number of clusters: 0, Number of noise points: 300
Eps: 0.2, MinPts: 3 - Number of clusters: 0, Number of noise points: 300
Eps: 0.3, MinPts: 3 - Number of clusters: 0, Number of noise points: 300
Eps: 0.4, MinPts: 3 - Number of clusters: 0, Number of noise points: 300
Eps: 0.5, MinPts: 3 - Number of clusters: 0, Number of noise points: 300
Eps: 0.6, MinPts: 3 - Number of clusters: 0, Number of noise points: 300
Eps: 0.7, MinPts: 3 - Number of clusters: 0, Number of noise points: 300
Eps: 0.8, MinPts: 3 - Number of clusters: 0, Number of noise points: 300
Eps: 0.9, MinPts: 3 - Number of clusters: 0, Number of noise points: 300
Eps: 1.0, MinPts: 3 - Number of clusters: 0, Number of noise points: 300
Eps: 1.1, MinPts: 3 - Number of clusters: 1, Number of noise points: 297
Eps: 1.2, MinPts: 3 - Number of clusters: 3, Number of noise points: 288
Silhouette Score for eps=1.2, MinPts=3: 0.6409
Eps: 1.3, MinPts: 3 - Number of clusters: 5, Number of noise points: 277
Silhouette Score for eps=1.3, MinPts=3: 0.4818
Eps: 1.4, MinPts: 3 - Number of clusters: 11, Number of noise points: 256
Silhouette Score for eps=1.4, MinPts=3: 0.4087
Eps: 1.5, MinPts: 3 - Number of clusters: 14, Number of noise points: 229
Silhouette Score for eps=1.5, MinPts=3: 0.3234
Eps: 1.6, MinPts: 3 - Number of clusters: 17, Number of noise points: 172
Silhouette Score for eps=1.6, MinPts=3: 0.1451
Eps: 1.7, MinPts: 3 - Number of clusters: 10, Number of noise points: 138
Silhouette Score for eps=1.7, MinPts=3: -0.0648
Eps: 1.8, MinPts: 3 - Number of clusters: 6, Number of noise points: 109
Silhouette Score for eps=1.8, MinPts=3: -0.0975
Eps: 1.9, MinPts: 3 - Number of clusters: 4, Number of noise points: 88
Silhouette Score for eps=1.9, MinPts=3: -0.0084
Eps: 2.0, MinPts: 3 - Number of clusters: 2, Number of noise points: 70
Silhouette Score for eps=2.0, MinPts=3: 0.1215
```

Figure 13: Output for the choice of eps and minpts.

Choice of MinPts:

Experiments with several MinPts values (2, 3, and 4) revealed the following: For MinPts = 2, the algorithm identified an excessive number of clusters, and the silhouette scores were consistently very high, potentially because the data had been overfitted (very small groups of points were recognized as valid clusters).

Valuable clusters started to form for MinPts = 3. The silhouette scores had greater balance and the noise points were lower with MinPts = 3 than with MinPts = 2.

It appears that this value could be too restrictive for the dataset with MinPts = 4, as the number of clusters drastically decreased to zero for the majority of Eps values.

Therefore, MinPts = 3 was selected since it strikes a balance between avoiding overfitting and maintaining meaningful clusters.

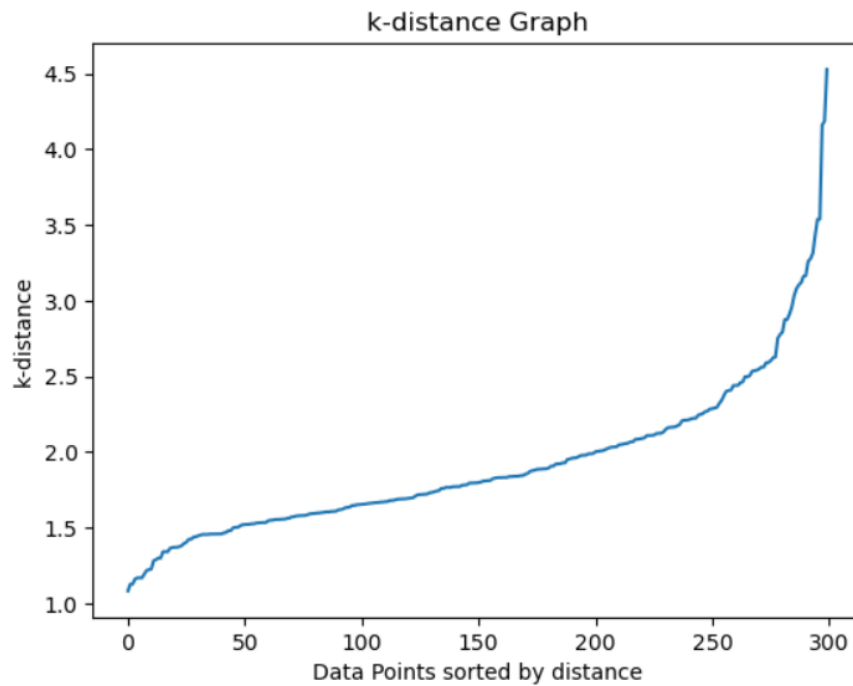


Figure 14: K-distance graph.

Choice of Eps:

Using the k-distance graph, the "elbow" point—where there is a notable increase in the distances between neighbors—was to determine the ideal value of Eps. The graph showed that distances naturally increased as Eps = 2.0 approached. Furthermore, testing with various Eps values revealed that the silhouette score was 0.1215 and the number of clusters was appropriate for Eps = 2.0. Therefore, MinPts = 3 was chosen because it gave a fair compromise between too few (or no) clusters and too granular clusters, and Eps = 2.0 was selected based on the k-distance graph's elbow point and the number of clusters formed, combined with a reasonable silhouette score.

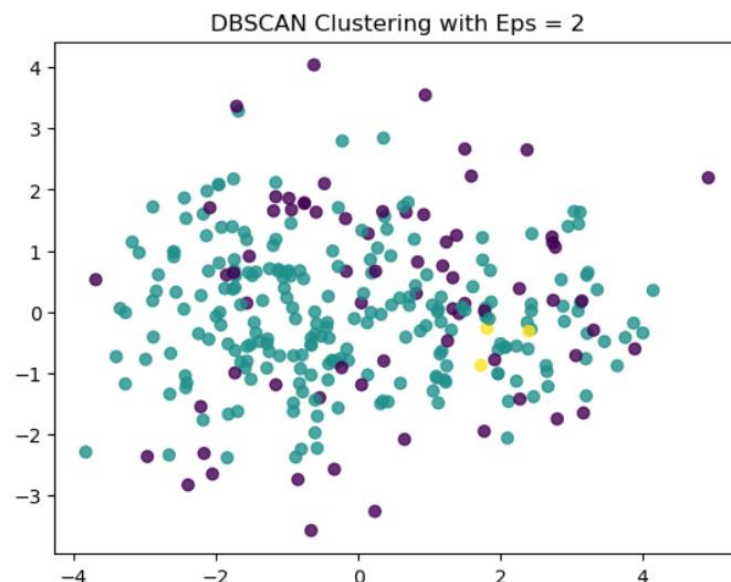


Figure 15: DBSCAN Clustering Plot.

Inference: Plotting demonstrates that DBSCAN is sensitive to density-based structures, as it has found two huge clusters with the required Eps value. A few of the points, especially the ones in yellow, have been labelled as noise, showing that DBSCAN handles low-density areas and outliers

better than k-means. DBSCAN does not make the spherical cluster assumption like k-means does. In this case, DBSCAN manages the clusters' apparent overlap and uneven forms well.

Performance Evaluation:

```
Silhouette Score: 0.1215      Adjusted Rand Index (ARI): 0.0135384981523833
Calinski-Harabasz Index: 6.1201  Normalized Mutual Information (NMI): 0.023054733865444273
Davies-Bouldin Index: 3.9388    Fowlkes-Mallows Index (FMI): 0.4668297047331011
```

Figure 16: Evaluation Metric results.

Inference: Several issues are indicated by the DBSCAN algorithm's performance. With numerous data points that are either poorly assigned or located near nearby clusters, the Silhouette Score of 0.1215 indicates a poor degree of cluster separation. The clusters are not compact or well-separated, as shown by the low Calinski-Harabasz Index (6.1201) and high Davies-Bouldin Index (3.9388), which point to overlapping or dispersed clusters.

While the low Normalized Mutual Information (0.0231) indicates little overlap between predicted clusters and actual labels, the Adjusted Rand Index (0.0135) demonstrates poor agreement between the DBSCAN clusters and true labels. Although the Fowlkes-Mallows Index (0.4668) shows a decent level of precision and recall, the overall clustering result is still inadequate. These metrics imply that DBSCAN was unable to successfully divide the dataset into meaningful clusters.

Comparison between K-Means and DBSCAN

```
k-Means Silhouette Score: 0.1590
DBSCAN Silhouette Score: 0.1215
k-Means Calinski-Harabasz Index: 65.0949
DBSCAN Calinski-Harabasz Index: 6.1201
k-Means Davies-Bouldin Index: 1.9644
DBSCAN Davies-Bouldin Index: 3.9388
```

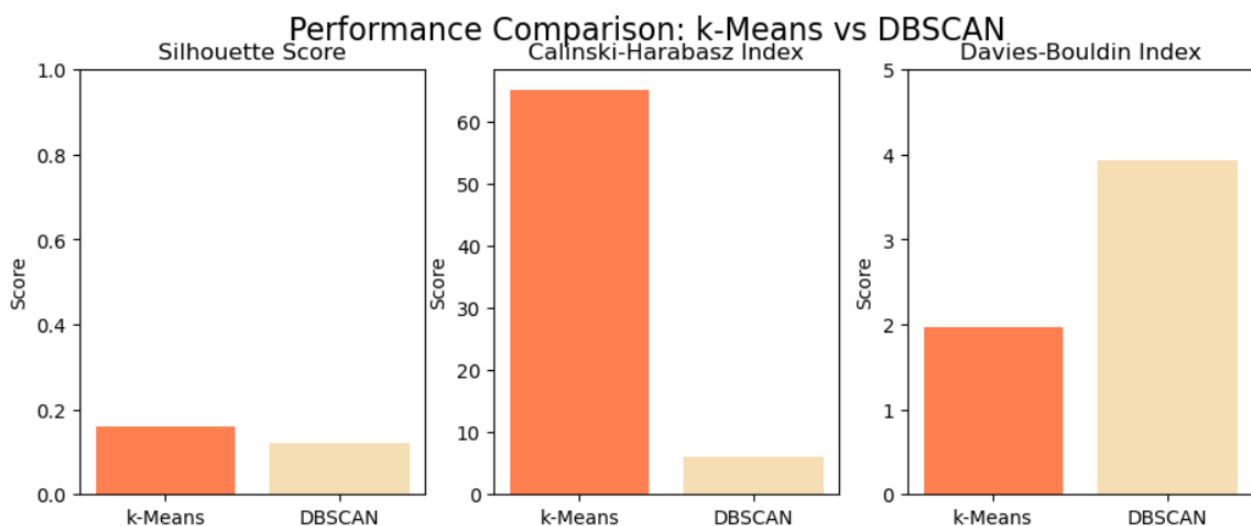


Figure 17: Comparison Plot of K-Means and DBSCAN.

Inference: There are significant differences in performance when k-Means and DBSCAN are compared using the Silhouette Score, Davies-Bouldin Index, and Calinski-Harabasz Index as the three clustering evaluation metrics.

Both k-Means (0.1590) and DBSCAN (0.1215) provide relatively low silhouette scores, indicating that neither technique fully separates clusters in this dataset. But overall, k-Means performs somewhat better than DBSCAN, indicating better-defined clusters. With a far higher Calinski-Harabasz Index (65.0949 compared to DBSCAN's 6.1201), k-Means performs significantly better than DBSCAN. This implies that compared to the clusters produced by DBSCAN, which seem to be

more dispersed, the clusters made by k-Means are more compact and well-separated. Better clustering performance is indicated by lower Davies-Bouldin Index values. In this case, k-Means (1.9644) outperforms DBSCAN (3.9388), suggesting that k-Means creates more distinctly separated clusters.

In summary, k-Means was the better-performing algorithm for this dataset based on the metrics used. Compared to DBSCAN, which had trouble drawing borders because of noise or poorly defined cluster densities, it created more compact, well-separated clusters. In situations when clusters are not spherical, DBSCAN's capacity to identify noise and create clusters of any shape is still advantageous, and it may still be applied to identify outliers that k-Means is unable to detect. Better results were obtained in this dataset because the k-Means assumption of spherical clusters aligned more closely with the underlying data distribution.

References

1. RMIT University. (2024). *[Week 5 Module]*. Retrieved from https://rmit.instructure.com/courses/125162/files/40605855/download?download_frd=1
2. RMIT University. (2024). *[Week 6 Module]*. Retrieved from https://rmit.instructure.com/courses/125162/files/40725423/download?download_frd=1
3. RMIT University. (2024). *[Week6-DecisionTreeExample]*. Retrieved from <https://rmit.instructure.com/courses/125162/modules/items/6451666>
4. RMIT University. (2024). *[Week 7 Module]*. Retrieved from <https://rmit.instructure.com/courses/125162/modules/items/6638409>
5. RMIT University. (2024). *[Week 8 Module]*. Retrieved from <https://rmit.instructure.com/courses/125162/modules/items/6649699>
6. RMIT University. (2024). *[DBSCAN.ipynb]*. Retrieved from <https://rmit.instructure.com/courses/125162/modules/items/6451684>
7. Sumony2j. (n.d.). *DBSCAN clustering on Wine Dataset* [GitHub repository]. GitHub. Retrieved from https://github.com/sumony2j/DBSCAN_Clustering/blob/main/Wine_Dataset_DBSCAN.ipynb
8. Shreyas3108. (n.d.). *House price prediction* [GitHub repository]. GitHub. Retrieved from <https://github.com/Shreyas3108/house-price-prediction/blob/master/housesales.ipynb>

Use of AI tools

Generative AI tools like ChatGPT was used for error handling and explanation. The for loop for different epsilon values was generated by the bot. The AI also recommended specific hyperparameters or exploration techniques. Apart from this AI wasn't used for anything else.